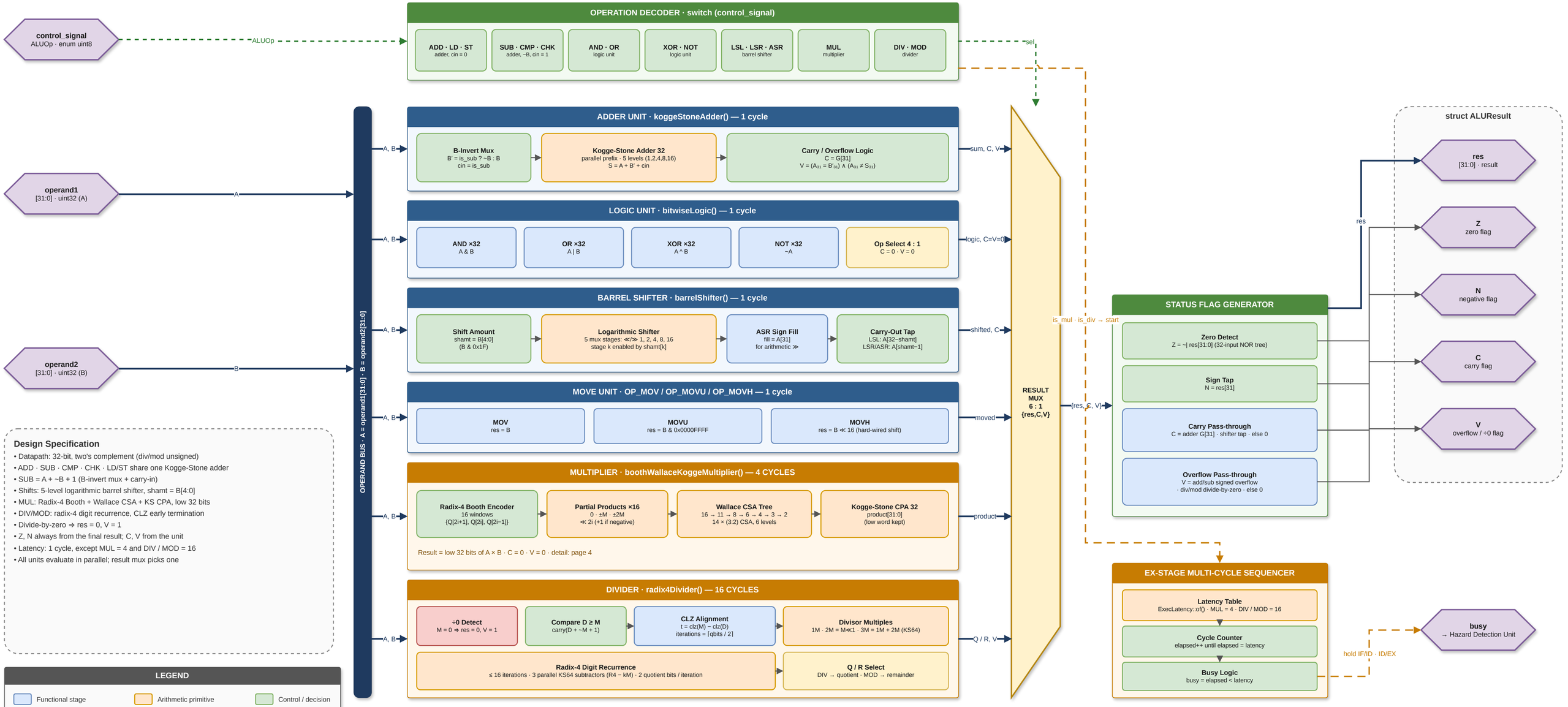
32-BIT INTEGER ALU — TOP-LEVEL HARDWARE ARCHITECTURE

ALU::execute() · operation decode · six parallel functional units · result mux · Z / N flag generation · struct ALUResult · multi-cycle MUL / DIV sequencing



Design Specification

- Datapath: 32-bit, two's complement (div/mod unsigned)
- ADD · SUB · CMP · CHK · LD/ST share one Kogge-Stone adder
- SUB = A + ~B + 1 (B-invert mux + carry-in)
- Shifts: 5-level logarithmic barrel shifter, shamt = B[4:0]
- MUL: Radix-4 Booth + Wallace CSA + KS CPA, low 32 bits
- DIV/MOD: radix-4 digit recurrence, CLZ early termination
- Divide-by-zero ⇒ res = 0, V = 1
- Z, N always from the final result; C, V from the unit
- Latency: 1 cycle, except MUL = 4 and DIV / MOD = 16
- All units evaluate in parallel; result mux picks one

LEGEND

Functional stage · Arithmetic primitive · Control / decision
Special-case logic · I/O port · Multiplexer
32-bit data path · Control signal
Iteration / multi-cycle · Special-case path

ADDER UNIT — koggeStoneAdder() DATAPATH

ADD · SUB · CMP · CHK · LD/ST address generation · B-invert + carry-in for subtraction · 32-bit Kogge-Stone parallel-prefix adder · carry-out & signed-overflow detection

operand1
[31:0] · A

operand2
[31:0] · B

Inverter ×32
~B

B-INV MUX 2 : 1

control_signal
ALUOp

Subtract Select
is_sub = SUB ∨ CMP ∨ CHK
ADD / LD / ST ⇒ is_sub = 0

A

B

~B

B′

cin

is_sub

ALUOp

PRE-PROCESSING · bitwise

Generate
$G_i = A_i \cdot B'_i$ (32 × AND)

Propagate
$P_i = A_i \oplus B'_i$ (32 × XOR)

Carry-in Merge
$G_0 = G_0 + P_0 \cdot cin$
(cin = is_sub)

Half-Sum Keep
H = A ⊕ B′ (re-used for final sum)

B′ = is_sub ? ~B : B

G, P

PARALLEL-PREFIX CARRY NETWORK · Kogge-Stone · $\log_2 32 = 5$ levels

Level 1 · d = 1
G |= P & (G ≪ 1)
P &= (P ≪ 1)
31 black cells
1 pass-through

G, P

Level 2 · d = 2
G |= P & (G ≪ 2)
P &= (P ≪ 2)
30 black cells
2 pass-through

G, P

Level 3 · d = 4
G |= P & (G ≪ 4)
P &= (P ≪ 4)
28 black cells
4 pass-through

G, P

Level 4 · d = 8
G |= P & (G ≪ 8)
P &= (P ≪ 8)
24 black cells
8 pass-through

G, P

Level 5 · d = 16
G |= P & (G ≪ 16)
(P not needed)
16 black cells
16 pass-through

Prefix operator ∘ (black cell)
$(G, P)_i \circ (G, P)_{i-d} = (G_i + P_i \cdot G_{i-d} , P_i \cdot P_{i-d})$ · bits i < d pass through unchanged (buffer)

After level 5
G[i] = group-generate of bits 0 … i (cin already folded into G_0) = carry out of bit i
Fan-out ≤ 2 per node · 129 black cells total · logic depth = 5 prefix levels
Critical path ≈ 1 (G/P) + 5 (prefix) + 1 (sum XOR) gate levels

G[31:0]

H

POST-PROCESSING

Carry Vector
c = (G ≪ 1) | cin

Sum XOR ×32
S = H ⊕ c = (A ⊕ B′) ⊕ c

Carry Out
C = G[31]
(SUB: C = 1 ⇔ no borrow)

Signed Overflow
$V = (A_{31} = B'_{31}) \wedge (A_{31} \neq S_{31})$
operands same sign,
result sign differs

S

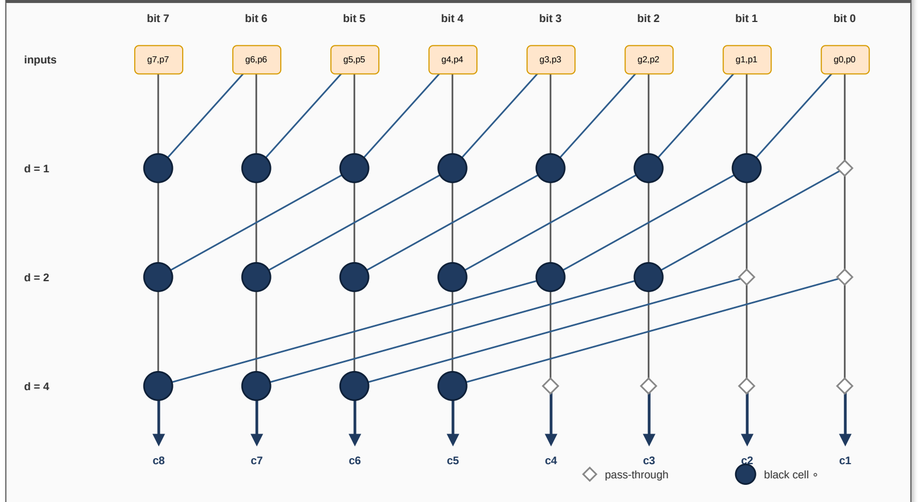
res
[31:0] · S

C
carry

V
overflow

Z, N added by ALU::execute()
(page 6)

KOGGE-STONE PREFIX GRAPH · 8-bit slice shown (32-bit adds levels d = 8, 16)



koggeStoneAdder64()
Same structure, 64-bit operands
6 prefix levels (d = 1 … 32)
cout = G[63]

Used by the divider for the 34-bit
partial remainder: R4 − kM and 3M = M + 2M

Operation Mapping

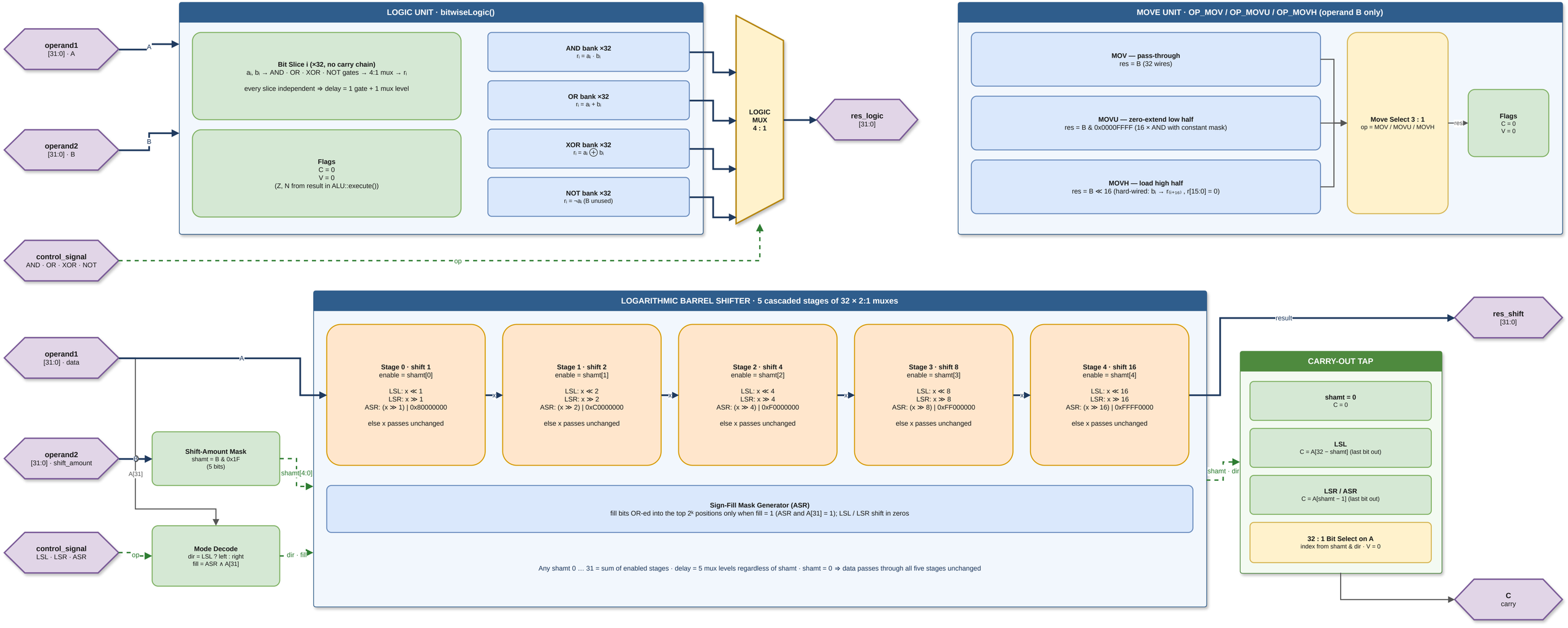
- ADD: B′ = B, cin = 0 ⇒ S = A + B
- LD / ST: S = base + imm (effective address)
- SUB: B′ = ~B, cin = 1 ⇒ S = A − B
- CMP: as SUB, only flags kept (E, GT)
- CHK: as SUB, A ≥ B (unsigned) ⇒ OUT_OF_BOUNDS
- Z, N set in ALU::execute() from S
- Single cycle · no pipeline registers

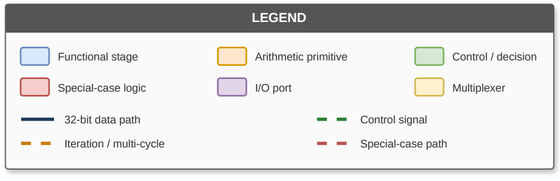
LEGEND

- Functional stage
- Arithmetic primitive
- Control / decision
- Special-case logic
- I/O port
- Multiplexer
- 32-bit data path
- Control signal
- Iteration / multi-cycle
- Special-case path

LOGIC UNIT · BARREL SHIFTER · MOVE UNIT — bitwiseLogic() / barrelShifter() DATAPATHS

32 identical bit slices for AND / OR / XOR / NOT · 5-stage logarithmic shifter (LSL · LSR · ASR) with sign fill and carry-out tap · MOV / MOVU / MOVH



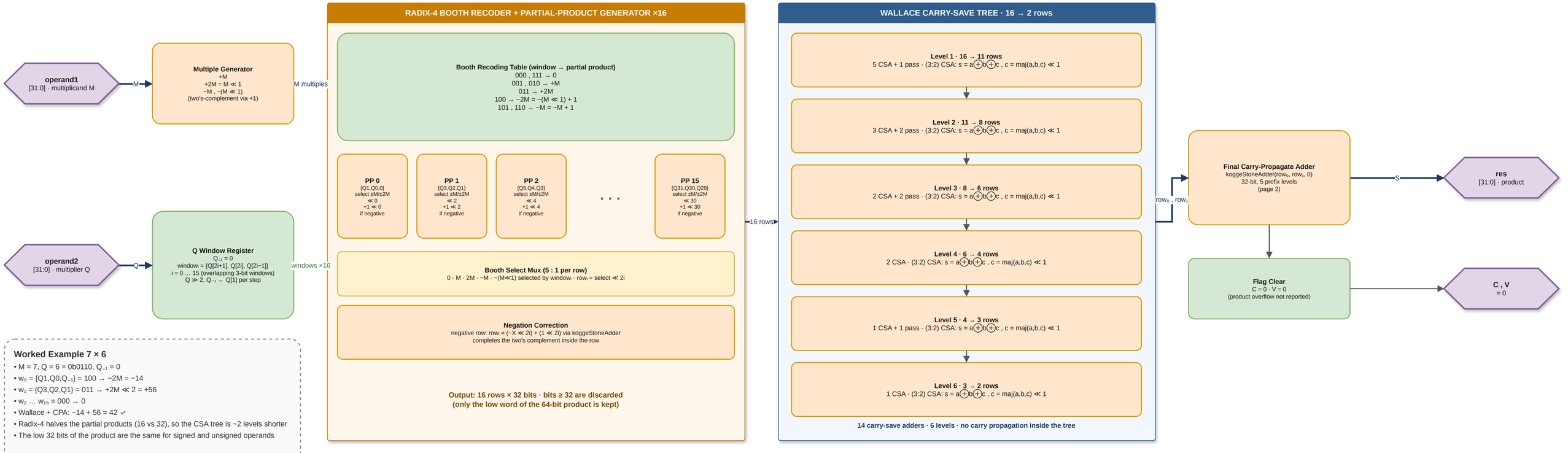


Shift Semantics (barrelShifter)

- Only B[4:0] is used: shifting by 32 or more wraps (shamt = B mod 32), matching barrelShifter()
- ASR replicates the original sign bit A[31] in every stage, so the fill is correct after any combination of stages
- Carry flag = the last bit shifted out; overflow is never set by shifts
- LSL by k = multiply by 2^k (mod 2^{32}) · LSR / ASR by k = unsigned / signed divide by 2^k (ASR rounds toward −∞)

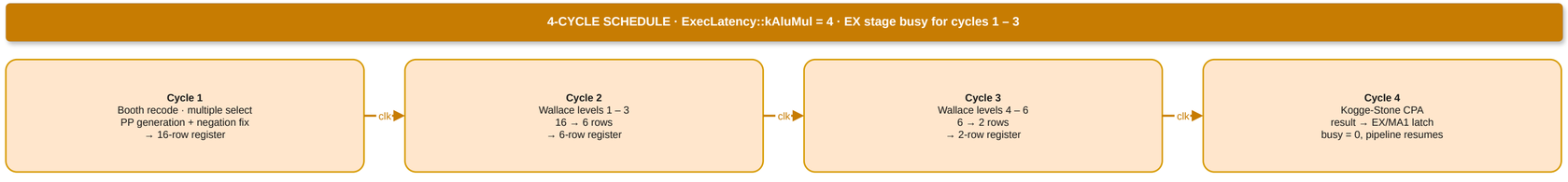
MULTIPLIER UNIT — boothWallaceKoggeMultiplier() DATAPATH

Radix-4 Booth recoding → 16 partial products · Wallace carry-save tree 16 → 2 · Kogge-Stone carry-propagate adder · low 32 bits · 4-cycle latency

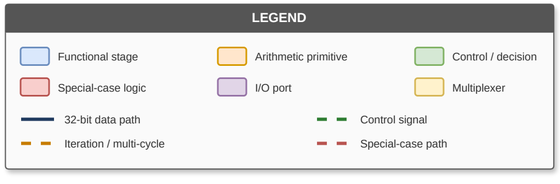


Worked Example 7 × 6

- M = 7, Q = 6 = 0b0110, Q_{-1} = 0
- w_0 = {Q1,Q0,Q_{-1}} = 100 → −2M = −14
- w_1 = {Q3,Q2,Q1} = 011 → +2M ≪ 2 = +56
- w_2 … w_{15} = 000 → 0
- Wallace + CPA: −14 + 56 = 42 ✓
- Radix-4 halves the partial products (16 vs 32), so the CSA tree is ~2 levels shorter
- The low 32 bits of the product are the same for signed and unsigned operands

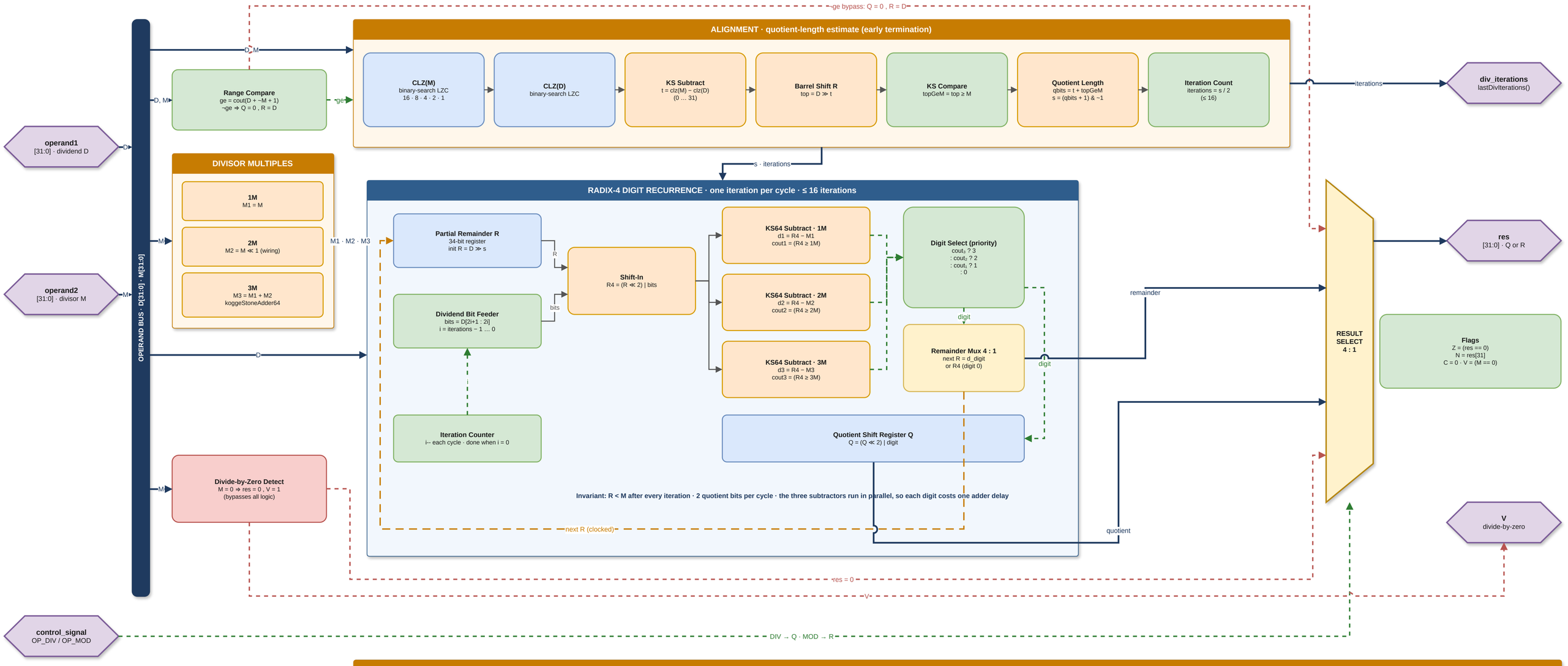


Suggested partition of the combinational datapath into the 4 EX cycles modelled by the simulator; the operands stay in the held ID/EX latch for the whole operation.



DIVIDER UNIT — radix4Divider() DATAPATH

Unsigned radix-4 digit recurrence · CLZ alignment with early termination · three parallel 34-bit subtractors per iteration · DIV → quotient, MOD → remainder · 16-cycle latency



LEGEND

- Functional stage
- Arithmetic primitive
- Control / decision
- Special-case logic
- I/O port
- Multiplexer
- 32-bit data path
- Control signal
- Iteration / multi-cycle
- Special-case path

16-CYCLE SCHEDULE · ExecLatency::kAluDiv = 16 · EX stage busy for cycles 1 – 15

Cycle 1
÷0 · compare · CLZ align · multiples
+ iteration 1

clk

Cycles 2 – 15
iterations 2 … 15 (2 quotient bits / cycle)
short quotients finish early; slot stays 16

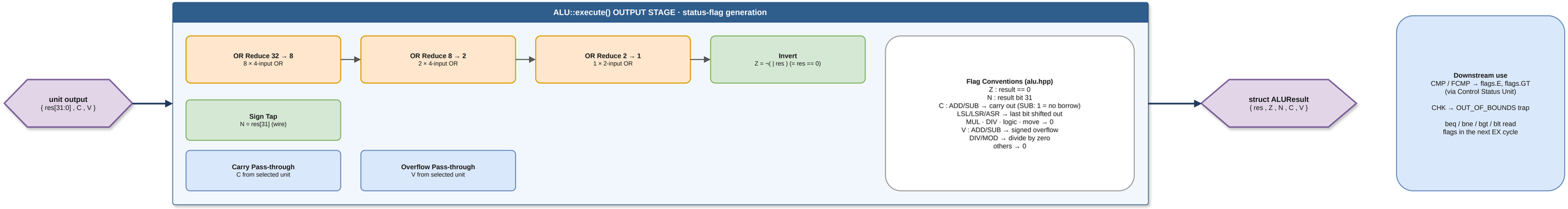
clk

Cycle 16
last iteration · Q / R select · flags
→ EX/MA1 latch, busy = 0

Worked example 100 ÷ 7
t = 4, top = 6 < 7 ⇒ s = 4, 2 iterations
R4 = 25 ≥ 3M → 3, R = 4 · R4 = 16 ≥ 2M → 2, R = 2
⇒ Q = 14 , R = 2 ✓

SHARED BLOCKS — STATUS-FLAG GENERATION AND ARITHMETIC PRIMITIVE LIBRARY

Z / N generation in ALU::execute() · flag conventions per operation · black-box primitives re-used by the adder, multiplier and divider



SHARED ARITHMETIC PRIMITIVE LIBRARY (black-box building blocks)

koggeStoneAdder32
a[31:0] , b[31:0] , cin → sum[31:0] , C , V
parallel prefix · 5 levels (1,2,4,8,16)

koggeStoneAdder64
a[63:0] , b[63:0] , cin → sum[63:0] , cout
parallel prefix · 6 levels (… ,32)

carrySaveAdder (3 : 2)
a , b , c → sum , carry
sum = $a \oplus b \oplus c$ · carry = maj(a,b,c) ≪ 1

countLeadingZeros32
x[31:0] → n[5:0]
binary search 16·8·4·2·1 · x = 0 ⇒ 32

barrelShifter
data[31:0] , shamt[4:0] , op → res , C
5 mux stages · ASR sign fill

bitwiseLogic
a , b , op → res
32 independent slices · AND OR XOR NOT

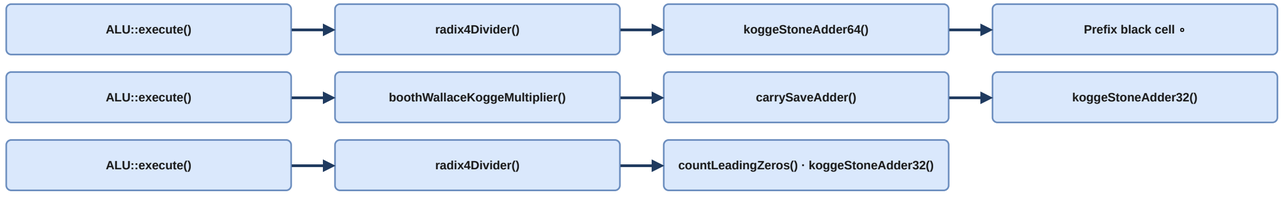
boothWallaceKoggeMultiplier
M , Q → product[31:0]
16 Booth PPs → 14 CSA → KS CPA

radix4Divider
D , M , op → Q or R , V
CLZ align · ≤ 16 radix-4 iterations

Prefix black cell ∘
$(G_h,P_h) , (G_l,P_l) \rightarrow (G_h + P_hG_l , P_hP_l)$
1 AND-OR + 1 AND

Full-adder bit cell
$a_i , b_i , c_i \rightarrow s_i , c_{i+1}$
building block of every CSA row

Instantiation hierarchy



Execute-Stage Latency Summary (src/headers/execution_latency.hpp)

1 cycle : ADD · SUB · CMP · CHK · LD/ST address · AND · OR · XOR · NOT · LSL · LSR · ASR · MOV · MOVU · MOVH

4 cycles : MUL (ExecLatency::kAluMul)

16 cycles : DIV · MOD (ExecLatency::kAluDiv)

While a multi-cycle op is in EX, the Hazard Detection Unit holds PC, IF/ID and ID/EX and sends bubbles downstream.

